

OS MIDTERM – Pocket Revision Sheet

/algorithmic-art /doc-coauthoring /all now make one docs with all answer in key point and make it fit in one paper and can be tear in small small part after print like small small sticky note make it in a way that it exactly fit in one paper in both side print a proper in samll font and all make so /all make in detail and fill all space like using all paper place

■ OS MIDTERM EXAM PREPARATION GUIDE

Operating Systems (OS)

This study guide contains complete, exam-standard descriptive answers for the core OS modules: Introduction, OS Structures, Processes, and CPU Scheduling.

■ Quick Jump

OS Midterm Topics

1. OS Functions & Multitasking

2. Dual Mode Operation

3. System Calls & API

4. OS Architectures

5. Process States

6. PCB & Context Switch

7. Scheduler Types

8. CPU First Principles

9. Numerical Masterclass

Q1. What is an Operating System? Detail its primary functions and compare Multiprogramming vs. Timesharing (Multitasking) systems.

1. Introduction to Operating System

An Operating System (OS) is a vital system software that acts as an intermediary or interface between a computer user and the underlying computer hardware. The primary goal of an operating system is to provide an environment in which a user can execute programs in a convenient and efficient manner. It abstracts the complex physical details of the hardware, making the computer system user-friendly, and controls, coordinates, and allocates hardware resources among various application programs for different users.

2. Primary Functions of an Operating System

The operating system manages the overall resources of the computer system. Its key functions include:

Process Management: The OS is responsible for creating, scheduling, executing, and deleting user and system processes. It manages mechanisms for process synchronization, communication, and deadlock handling.

Memory Management: The OS tracks which parts of memory are currently in use and by whom. It dynamically allocates and de-allocates memory blocks to processes as they request and release them.

File (Storage) Management: The OS abstracts physical storage devices to present logical directories and files. It handles operations like file creation, deletion, modification, directory mapping, and backup.

I/O Device Management: The OS masks the differences of device drivers by providing a uniform device-access interface. It manages buffering, caching, spooling, and driver operations.

Protection and Security: The OS ensures that resources are protected against unauthorized access. It defines user permissions, authentication mechanisms, and safeguards memory boundaries.

3. Comparison: Multiprogramming vs. Timesharing (Multitasking)

While both techniques aim to improve CPU utilization, they operate on different philosophies:

Comparison Factor Multiprogramming Systems Timesharing (Multitasking) Systems

Core Objective To maximize CPU utilization. Keep the CPU busy at all times by having multiple jobs in memory. To minimize response time. Provide interactive, rapid sharing of the CPU among multiple users/tasks.

Switching Mechanism The CPU switches to another job only when the current job goes for an I/O operation. The CPU switches between jobs at regular intervals defined by a time quantum (time slice), even if the job isn't waiting for I/O.

User Interaction Non-interactive. Users do not interact with the system during program execution. Highly interactive. Users get immediate feedback (usually in milliseconds), giving the illusion of dedicated machine access.

Key Concept Job scheduling and memory pooling. Time-slicing and context switching.

Exam Tip: Remember that Timesharing is a direct logical extension of Multiprogramming. While Multiprogramming maximizes hardware throughput, Timesharing optimizes human productivity and user experience.

Q2. Explain the Dual-Mode Operation (User Mode vs. Kernel Mode) in Operating Systems. Why is it necessary, and how does the hardware transition between modes?

1. The Concept of Dual-Mode Operation

To ensure the proper and secure operation of the computer system, the operating system and hardware must protect themselves against malicious or poorly written user programs. This is achieved through

Dual-Mode Operation, which divides CPU execution into two distinct operating environments:

User Mode (Mode Bit = 1): This is the restricted mode in which user applications execute. User programs cannot directly access hardware resources or read/write to protected memory spaces. They must request the OS kernel to perform these operations on their behalf.

Kernel Mode (Mode Bit = 0): Also known as Supervisor, System, or Privileged Mode. This is the mode in which the core OS kernel executes. The CPU has unrestricted access to the entire instruction set, all hardware registers, and all memory locations.

2. Why Dual-Mode is Necessary

Without dual-mode operation, a user program could crash the entire operating system, overwrite critical kernel instructions in memory, or access data belonging to other users. For example, if a user program could execute a direct physical write command to the hard disk without the kernel's mediation, it could corrupt files belonging to other users or bypass security checks. Privileged instructions (like disabling interrupts, modifying the page tables, or writing to input/output devices) can only be executed in Kernel Mode.

3. The Transition Mechanism (User to Kernel and Back)

The hardware transition between modes is mediated by a physical Mode Bit in the CPU status register. When a computer starts, it boots in Kernel Mode. The OS loads, sets up the environment, and switches the mode bit to 1 (User Mode) before handing control to a user application.

Dual-Mode Transition Process

[User Application] -> Executes user instructions (Mode Bit = 1)

|
+----> Issues System Call (e.g. Read File) -> Generates TRAP / Interrupt

|
V
[Hardware Logic] -> Changes Mode Bit to 0 (Kernel Mode)

|
V
[OS Kernel Service Routine] -> Executes privileged operations

|
V
[Hardware Logic] -> Resets Mode Bit to 1 (User Mode)

|
V
[User Application] <-- Returns result from System Call

A transition occurs whenever a system call is made, or when an interrupt or system trap is triggered. The hardware automatically flips the Mode Bit to 0, jumps to a pre-defined instruction address inside the kernel (Interrupt Vector), handles the request safely, flips the bit back to 1, and returns execution control back to the user space.

Q3. What is a System Call? How does it differ from a standard API function? Describe the three methods used to pass parameters to the OS during a system call.

1. Definition: System Call vs. API

A System Call is the programming interface that allows user-level programs to request services from the operating system kernel. It is the only entry point into the kernel from user space. Since user programs are restricted by dual-mode operation, any task requiring hardware interaction (such as displaying text on a monitor, writing to disk, or creating a process) must be requested via a system call.

An Application Programming Interface (API), on the other hand, is a set of functions and protocols available to the programmer. Programming directly in system calls is highly complex and platform-specific.

Therefore, developers write code against standard APIs (like POSIX API in Unix/Linux or Win32 API in Windows) rather than calling raw system calls. The API library function automatically intercepts the programmer's request, formats the arguments, and executes the underlying system call on behalf of the application.

2. Methods of Parameter Passing

When a system call is invoked, the user program often needs to pass parameters (such as file names, data buffers, or sizes) to the operating system kernel. Operating systems use three primary methods to pass these parameters:

Method 1: Pass-by-Registers (Register Method)

The simplest and fastest method is to write parameters directly into the CPU's registers. When the CPU trap occurs, the kernel reads parameters straight out of those registers.

Limitation: It is limited by the number and size of CPU registers. If there are more parameters than available registers, this method fails.

Method 2: Pass-by-Block/Table (Memory Block Method)

Parameters are written into a contiguous block or table in memory. The memory address of this block is then loaded into a single CPU register, which is passed to the kernel. When the kernel takes over, it reads the address from the register and fetches all parameters directly from RAM.

Advantage: Highly scalable. It allows passing an arbitrary number of parameters of any size. This is the preferred method used by UNIX and Linux.

Method 3: Pass-by-Stack (Stack Method)

The user program pushes the parameters onto its user stack. When the system call instruction executes, the kernel pops the parameters off the stack to use them.

Advantage: Safe and clean structure, avoiding register limitations and separating data blocks cleanly.

Q4. Compare the structural architectures of Operating Systems: Monolithic (Simple), Layered, Microkernel, and Modular (LKM). Outline the trade-offs of each.

1. Monolithic Structure

In a Monolithic Architecture, the entire operating system operates as a single, massive, cohesive program running completely in Kernel Mode. All OS services share the same memory address space.

Trade-offs: Excellent execution speed and low system overhead because services communicate via fast direct function calls. However, it lacks modularity. A crash in any single service can corrupt kernel memory and crash the entire system. It is also difficult to modify and maintain.

2. Layered Approach

The OS is split into hierarchically numbered levels or Layers. Layer 0 represents the hardware, while the highest Layer N represents the user interface. Each layer can only invoke functions from layers directly below it.

Trade-offs: Exceptionally easy to debug and verify. However, designing layers cleanly is extremely difficult, and it suffers from poor performance because an I/O request must travel sequentially through multiple layers.

3. Microkernel Architecture

The Microkernel approach minimizes the kernel by stripping out all non-essential components and running them as user-space programs called servers. The microkernel itself handles only basic IPC, memory management, and low-level CPU scheduling.

Trade-offs: Excellent security and reliability. If a server crashes, the system remains up. However, it suffers from heavy performance degradation because every communication between servers requires switching modes and copying messages across user-kernel boundaries.

4. Modular Approach (Loadable Kernel Modules - LKM)

The modern standard. The kernel retains a core set of components and dynamically loads or unloads additional components (modules) at runtime as needed.

Trade-offs: It combines the speed of monolithic architectures (modules run directly in kernel space once loaded) with the flexibility and safety of the microkernel approach.

Q5. What is a Process? Explain the Process State Transition Diagram with all five states in detail.

1. Defining a Process

A Process is formally defined as a "program in execution." A process's memory layout consists of four distinct sections:

Text Section: Stores the executable code instructions.

Data Section: Contains global and static variables.

Heap Section: Dynamically allocated memory during runtime.

Stack Section: Temporary data storage (function parameters, return addresses, local variables).

2. The Process State Transition Diagram

As a process executes, it moves through different states. The standard five-state model is illustrated below:

Standard 5-State Process Model

New

Created

➔

Ready

Waiting CPU

■

Running

Executing

➔

Terminated

Finished

From Running:

➔

Waiting

I/O or Event

➔ (Event Occurs) ➔

Ready

3. Detailed Explanation of the States

New: The process is currently being created.

Ready: The process is loaded in main memory and is waiting to be allocated CPU execution time.

Running: The CPU scheduler has dispatched the process, and its instructions are executing on a CPU core.

Waiting (Blocked): The process cannot execute until some specific event occurs (such as I/O completion or signal).

Terminated: The process has completed its execution, and its resources are reclaimed.

Q6. Define Process Control Block (PCB). Detail its key components and illustrate the steps involved in a Context Switch.

1. What is a Process Control Block (PCB)?

A Process Control Block (PCB) is a data structure maintained by the operating system for every active process. It serves as the repository of all information needed to track, manage, and execute the process.

2. Key Components of a PCB

Process State: The current state of the process (New, Ready, Running, etc.).

Program Counter (PC): Indicates the address of the next instruction to execute.

CPU Registers: Saves the contents of CPU accumulators, registers, and stack pointers when the process is suspended.

CPU Scheduling Information: Process priority and scheduling queue pointers.

Memory-Management Information: Stores page tables or segment tables.

I/O Status Information: Lists open files and allocated devices.

3. Context Switch Mechanism

A Context Switch is the process of saving the execution state of the currently running process and restoring the saved execution state of a different process. This allows the CPU to switch execution without losing state, representing system overhead since no user work is completed during the switch.

Q7. Differentiate between Short-term, Medium-term, and Long-term Schedulers. Explain the role of CPU Scheduling Queues.

1. The Role of Schedulers

Schedulers manage process selection and transitions. An operating system uses three distinct types:

Comparison Parameter Long-Term Scheduler (Job Scheduler) Short-Term Scheduler (CPU Scheduler) Medium-Term Scheduler (Swapper)

Primary Role Selects processes from disk and loads them into memory. Selects processes from ready queue and allocates CPU. Swaps processes out of memory to disk, and back.

Execution Frequency Infrequent (seconds or minutes). Extremely frequent (milliseconds). Moderate (triggered during memory pressure).

Degree of Multiprogramming Controls the number of processes in memory. Does not control the degree of multiprogramming. Dynamically decreases the degree of multiprogramming.

Q8. CPU Scheduling First Principles: Core Concepts, Variables, Equations, and Ready Queue Tracing Mechanics.

1. First Principles of Scheduling Mathematics

CPU Scheduling solves the mathematical problem of ordering processes for CPU execution to optimize system parameters. Before solving any problem, you must master the fundamental formulas derived from first principles:

■ Core Mathematical Definitions

Arrival Time (AT): The exact physical timestamp when a process enters the Ready Queue.

Burst Time (BT): The exact CPU execution time required by the process to complete its task.

Completion Time (CT): The timestamp when a process finishes its final instruction. (Obtained directly from the right-hand edge of the process block in the Gantt chart).

Turnaround Time (TAT): The total duration a process spends in the system from arrival to completion.

TAT = Completion Time (CT) - Arrival Time (AT)

Waiting Time (WT): The total cumulative time a process spends waiting in the Ready Queue without CPU allocation.

WT = Turnaround Time (TAT) - Burst Time (BT)

Response Time (RT): The duration from arrival to the first time CPU execution begins.

RT = First CPU Allocation Time - Arrival Time (AT)

2. First Principles of the Ready Queue (FIFO Logic)

The **Ready Queue** is a dynamic list of processes waiting to be allocated to the CPU. In a preemptive system or Round Robin system, how and when processes enter the queue determines the exact output.

You must follow these strict priority rules for queue insertion:

New Arrivals: When a process arrives at time \$t\$, it is immediately appended to the end of the Ready Queue.

Preempted Processes: If a running process is preempted (e.g., its time quantum expires) at time \$t\$, it is removed from the CPU.

Tie-Breaking Rule: If a new process arrives at time \$t\$ AND a running process is preempted at the exact same time \$t\$, the new arrival is inserted first, and then the preempted process is appended behind it.

Q9. CPU Scheduling Numerical Masterclass: Complete Step-by-Step Walkthroughs for All 6 Types from First Principles.

We will solve every numerical scheduling algorithm type using the exact same underlying mathematical process. Pay close attention to the Ready Queue traces and calculations.

Type 1: First-Come, First-Served (FCFS) — Different Arrival Times

Rule: The CPU executes processes in the order they arrive. It is strictly non-preemptive.

Process Arrival Time (AT) Burst Time (BT)

P1 0 4

P2 1 3

P3 2 1

P4 4 2

Ready Queue Tracing:

Time 0: P1 arrives. Ready Queue: [P1]. CPU executes P1 (0 to 4).

Time 1: P2 arrives. Ready Queue: [P2].

Time 2: P3 arrives. Ready Queue: [P2, P3].

Time 4: P1 completes. P4 arrives. Ready Queue: [P2, P3, P4]. CPU selects P2 from head of queue and executes (4 to 7).

Time 7: P2 completes. Ready Queue: [P3, P4]. CPU executes P3 (7 to 8).

Time 8: P3 completes. Ready Queue: [P4]. CPU executes P4 (8 to 10).

Gantt Chart:

P1

P2

P3

P4

0

4

7

8

10

Mathematical Calculations:

Process AT BT CT TAT (CT-AT) WT (TAT-BT)

P1 0 4 4 0

P2 1 3 7 3

P3 2 1 6 5

P4 4 2 10 6 4

Average TAT = (4 + 6 + 6 + 6) / 4 = 5.50 units

Average WT = (0 + 3 + 5 + 4) / 4 = 3.00 units

Type 2: Non-preemptive Shortest Job First (SJF) — Different Arrival Times

Rule: Once CPU is assigned, the process runs to completion. When selecting the next process, choose the one in the Ready Queue with the smallest Burst Time.

Process Arrival Time (AT) Burst Time (BT)

P1 0 7
P2 2 4
P3 4 1
P4 5 4
Ready Queue Tracing:
Time 0: P1 arrives. Ready Queue: [P1]. CPU executes P1 (0 to 7).
Time 2: P2 arrives. Ready Queue: [P2].
Time 4: P3 arrives. Ready Queue: [P2, P3].
Time 5: P4 arrives. Ready Queue: [P2, P3, P4].
Time 7: P1 completes. We check the Ready Queue [P2, P3, P4] and sort by Burst Time: P3 (BT=1), P2 (BT=4), P4 (BT=4). P3 has the smallest BT. CPU executes P3 (7 to 8).
Time 8: P3 completes. Ready Queue is [P2, P4]. Both have BT=4. We break the tie using Arrival Time: P2 arrived first (at 2). CPU executes P2 (8 to 12).
Time 12: P2 completes. Ready Queue is [P4]. CPU executes P4 (12 to 16).

Gantt Chart:
P1
P3
P2
P4
0
7
8
12
16
Mathematical Calculations:
Process AT BT CT TAT (CT-AT) WT (TAT-BT)
P1 0 7 7 7 0
P2 2 4 12 10 6
P3 4 1 8 4 3
P4 5 4 16 11 7
Average TAT = (7 + 10 + 4 + 11) / 4 = 8.00 units
Average WT = (0 + 6 + 3 + 7) / 4 = 4.00 units
Type 3: Preemptive Shortest Job First (Shortest Remaining Time First — SRTF)
Rule: The system evaluates execution at every single arrival event. If a newly arrived process has a Burst Time shorter than the remaining execution time of the currently running process, preempt the CPU immediately.

Process Arrival Time (AT) Burst Time (BT)
P1 0 8
P2 1 4
P3 2 9
P4 3 5
Ready Queue Tracing (Evaluated millisecond by millisecond):
Time 0: P1 arrives. BT=8. CPU runs P1. (Remaining BT: P1=8)
Time 1: P2 arrives. BT=4.
Compare remaining P1 (7) vs P2 (4). Since P2 is shorter, preempt P1.
CPU executes P2. (Remaining BT: P1=7, P2=4)
Time 2: P3 arrives. BT=9.
Compare remaining P2 (3) vs P3 (9) vs P1 (7). P2 is shortest, CPU continues P2.
(Remaining BT: P1=7, P2=3, P3=9)
Time 3: P4 arrives. BT=5.
Compare remaining P2 (2) vs P4 (5) vs P1 (7) vs P3 (9). P2 is shortest, CPU continues P2.
(Remaining BT: P1=7, P2=2, P3=9, P4=5)
Time 5: P2 completes execution.
Active pool in memory: P1 (7), P3 (9), P4 (5).
Smallest remaining is P4 (BT=5). CPU executes P4 (5 to 10).
Time 10: P4 completes execution.
Active pool: P1 (7), P3 (9).
Smallest remaining is P1 (BT=7). CPU executes P1 (10 to 17).
Time 17: P1 completes execution.
Active pool: P3 (9).
CPU executes P3 (17 to 26).
Time 26: P3 completes.

Gantt Chart:
P1
P2
P4
P1
P3
0
1
5
10
17
26
Mathematical Calculations:
Process AT BT CT TAT (CT-AT) WT (TAT-BT)
P1 0 8 17 17 9
P2 1 4 5 4 0
P3 2 9 26 24 15
P4 3 5 10 7 2
Average TAT = (17 + 4 + 24 + 7) / 4 = 13.00 units
Average WT = (9 + 0 + 15 + 2) / 4 = 6.50 units
Type 4: Non-Preemptive Priority Scheduling — Different Arrival Times
Rule: The CPU runs the current process to completion. When selecting the next process, choose the one with the highest priority (in this example, a smaller number represents a higher priority).

Process Arrival Time (AT) Burst Time (BT) Priority
P1 0 3 2
P2 1 4 3
P3 2 2 1
Ready Queue Tracing:
Time 0: P1 arrives. Ready Queue: [P1]. CPU executes P1 (0 to 3).
Time 1: P2 arrives. Ready Queue: [P2].
Time 2: P3 arrives. Ready Queue: [P2, P3].
Time 3: P1 completes. Ready Queue is [P2 (Pri=3), P3 (Pri=1)]. P3 has a higher priority (1 < 3). CPU executes P3 (3 to 5).
Time 5: P3 completes. Ready Queue is [P2]. CPU executes P2 (5 to 9).

Gantt Chart:
P1
P3
P2
0
3
5
9
Mathematical Calculations:
Process AT BT CT TAT (CT-AT) WT (TAT-BT)
P1 0 3 3 3 0
P2 1 4 9 8 4
P3 2 2 5 3 1
Average TAT = (3 + 8 + 3) / 3 = 4.67 units
Average WT = (0 + 4 + 1) / 3 = 1.67 units
Type 5: Preemptive Priority Scheduling — Different Arrival Times
Rule: Evaluate the queue at every arrival event. If a newly arrived process has a higher priority than the currently running process, preempt the CPU immediately.
Process Arrival Time (AT) Burst Time (BT) Priority (Smaller = Higher)
P1 0 4 2
P2 1 3 1
P3 2 2 3

Ready Queue Tracing:
Time 0: P1 arrives. Priority=2. CPU executes P1.
Time 1: P2 arrives. Priority=1.
Compare current P1 (Pri=2) vs new P2 (Pri=1). P2 has higher priority, so preempt P1.
CPU executes P2. (Remaining BT: P1=3, P2=3)
Time 2: P3 arrives. Priority=3.
Compare current P2 (Pri=1) vs P1 (Pri=2) vs P3 (Pri=3). P2 remains highest, CPU continues P2.
(Remaining BT: P1=3, P2=2, P3=2)
Time 4: P2 completes execution.
Active pool in memory: P1 (Pri=2, BT=3) and P3 (Pri=3, BT=2).
Highest priority is P1. CPU executes P1 (4 to 7).
Time 7: P1 completes execution.
Active pool: P3.
CPU executes P3 (7 to 9).
Time 9: P3 completes.

Gantt Chart:

P1

P2

P1

P3

0

1

4

7

9

Mathematical Calculations:

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	4	7	7	3
P2	1	3	4	3	0
P3	2	2	9	7	5

Average TAT = $(7 + 3 + 7) / 3 = 5.67$ units
Average WT = $(3 + 0 + 5) / 3 = 2.67$ units
Type 6: Round Robin (RR) — Different Arrival Times (Ultimate Tracing Challenge)
Rule: Every process receives a maximum execution slot of Time Quantum (TQ). If a process doesn't finish, it goes to the end of the Ready Queue. We follow strict FIFO queue insertion rules when process preemption and new arrivals occur simultaneously.

■ First Principles Queue Rule: At the exact millisecond a process is preempted, any new processes arriving at that exact millisecond must enter the ready queue first. The preempted process is appended after them.

Process Arrival Time (AT) Burst Time (BT)

P1

0

5

P2

1

3

P3

2

1

P4

3

2

P5

4

3

Time Quantum = 2 units.

Ready Queue Step-by-Step Chronological Trace:

Time 0: P1 arrives. Ready Queue: [P1]
CPU schedules P1 (0 to 2). (P1 remaining BT: 3)
Time 1: P2 arrives. P2 enters Ready Queue.
Ready Queue state: [P2]
Time 2: P3 arrives.
P1's quantum expires at 2. P1 is preempted.
According to FIFO rules, we first add new arrivals at 2 (P3), then append preempted (P1).
Ready Queue state: [P2, P3, P1]
CPU schedules P2 (2 to 4). (P2 remaining BT: 1)
Time 3: P4 arrives. P4 enters Ready Queue.
Ready Queue state: [P3, P1, P4]
Time 4: P5 arrives.
P2's quantum expires at 4. P2 is preempted (remaining BT: 1).
We first add new arrivals at 4 (P5), then append preempted (P2).
Ready Queue state: [P3, P1, P4, P5, P2]
CPU schedules P3 (4 to 5). P3 finishes execution at 5.
Time 5: Ready Queue state: [P1, P4, P5, P2]
CPU schedules P1 (5 to 7). (P1 remaining BT: 1)
Time 7: P1's quantum expires at 7. P1 is preempted.
Ready Queue state: [P4, P5, P2, P1]
CPU schedules P4 (7 to 9). P4 finishes execution at 9 (since its BT was 2).
Time 9: Ready Queue state: [P5, P2, P1]
CPU schedules P5 (9 to 11). (P5 remaining BT: 1)
Time 11: P5's quantum expires at 11. P5 is preempted.
Ready Queue state: [P2, P1, P5]
CPU schedules P2 (11 to 12). P2 finishes execution at 12 (since remaining BT was 1).
Time 12: Ready Queue state: [P1, P5]
CPU schedules P1 (12 to 13). P1 finishes execution at 13 (since remaining BT was 1).
Time 13: Ready Queue state: [P5]
CPU schedules P5 (13 to 14). P5 finishes execution at 14 (since remaining BT was 1).
Time 14: Queue is empty. All processes complete.

Gantt Chart:

P1

P2

P3

P1

P4

P5

P2

P1

P5

0

2

4

5

7

9

11

12

13

14

Mathematical Calculations:

Process	ID	AT	BT	CT	Turnaround Time (CT-AT)	Waiting Time (TAT-BT)
P1	0	5	13	13	8	
P2	1	3	12	11	8	
P3	2	1	5	3	2	
P4	3	2	9	6	4	
P5	4	3	14	10	7	

Average Turnaround Time = $(13 + 11 + 3 + 6 + 10) / 5 = 43 / 5 = 8.60$ units
Average Waiting Time = $(8 + 8 + 2 + 4 + 7) / 5 = 29 / 5 = 5.80$ units

like the way of image